

```

//+-----
//|                                     RectangleZoneAnalyzer_06.mq4
//|
//|  Блок 06: настоящий rolling walk-forward без отбора на будущем.
//|
//|  Вход : touches.csv
//|
//|  Выход: pattern_true_walkforward.csv
//|          pattern_true_walkforward_summary.csv
//|
//|  По умолчанию:
//|      сегменты 1-4 -> поиск/отбор, сегмент 5 -> OOS;
//|      сегменты 2-5 -> новый поиск/отбор, сегмент 6 -> OOS.
//|
//|  Важная защита от будущего:
//|  результат касания учитывается в обучении/OOS только тогда,
//|  когда время результата не выходит за конец соответствующего
//|  окна. MFE/MAE используются только при полностью завершённом
//|  локальном наблюдении внутри окна.
//+-----
#property copyright "Copyright 2026"
#property version  "1.00"
#property strict
#property script_show_inputs

//--- Файлы находятся в MQL4\Files
input string InputTouchesFileName           =
"touches.csv";
input string OutputWalkForwardFileName      =
"pattern_true_walkforward.csv";
input string OutputWalkForwardSummaryFileName =
"pattern_true_walkforward_summary.csv";

```

```

//--- Анализ текущего символа и таймфрейма
input bool    FilterCurrentChartOnly          = true;

//--- Разбиение истории и rolling-окно
input int     NumberOfSegments                = 6;
input int     TrainingSegments                = 4;

//--- Условия отбора паттерна только на обучающем окне
input int     MinTrainSamples                 = 100;
input int     StrongTrainSamples              = 300;
input int     MinTrainLocalDecisive           = 80;
input int     MinTrainStructuralDecisive      = 80;
input double  MinTrainLocalWilson             = 25.0;
input double  MinTrainStructuralWilson        = 50.0;
input double  MinTrainImprovementPctPoints    = 2.0;
input double  MinTrainMfeMaeRatio            = 0.0;

//--- Веса обучающего рейтинга
input double  WeightLocalWilson               = 0.35;
input double  WeightStructuralWilson          = 0.40;
input double  WeightMfeMae                   = 0.10;
input double  WeightImprovement              = 0.15;

//--- 0 = сохранить все отобранные паттерны каждого окна
input int     MaxSelectedPatternsPerFold      = 0;

//--- Минимальная наполненность независимого OOS-сегмента
input int     MinOosSamples                   = 20;
input int     MinOosLocalDecisive             = 15;
input int     MinOosStructuralDecisive        = 15;
input double  MinOosImprovementPctPoints      = 0.0;

//--- Границы торговых сессий, как в блоках 03-05
input int     AsiaEndHour                     = 7;
input int     LondonEndHour                   = 13;
input int     NewYorkEndHour                  = 21;

//--- Печать прогресса; 0 отключает
input int     ProgressEveryRows               = 5000;

#define TOUCHES_CSV_COLUMNS 137
#define ZONE_COUNT          2
#define AGE_COUNT           9
#define TOUCH_BUCKET_COUNT  5
#define DEPTH_BUCKET_COUNT  4
#define APPROACH_COUNT       4
#define HEIGHT_COUNT        5
#define SESSION_COUNT        4

```

```

//+-----

```

```
///  
//| Статистика набора касаний
```

```
///  
//+-----
```

```
struct SampleStat  
{  
    int samples;  
  
    int localReversal;  
    int localBreakout;  
  
    int structuralOpposite;  
    int structuralSameType;  
  
    double sumMFE;  
    int countMFE;  
    double sumMAE;  
    int countMAE;  
};
```

```
///  
//+-----
```

```
///  
//| Формальное описание исследуемого паттерна
```

```
///  
//+-----
```

```
struct CandidateDef  
{  
    string patternType;  
  
    int zoneIndex;  
    int ageIndex;  
    int touchIndex;  
    int depthIndex;  
    int approachIndex;  
    int heightIndex;  
    int sessionIndex;  
  
    string patternID;  
};
```

```
///  
//+-----
```

```
///  
//| Отобранный паттерн одного обучающего окна
```

```
///  
//+-----
```

```
struct FoldSelection  
{  
    int foldIndex;  
    int candidateIndex;  
    int parentIndex;  
};
```

```

int foldRank;

string selectionReason;
string reliabilityTier;

double trainLocalWilson;
double trainStructuralWilson;
double parentLocalWilson;
double parentStructuralWilson;
double localImprovement;
double structuralImprovement;

double sampleFactor;
double payoffScore;
double improvementScore;
double finalScore;
};

//+-----

//| Итог одного паттерна по независимым окнам

//+-----
struct PatternSummary
{
    int candidateIndex;
    int selectedFolds;
    int eligibleFolds;
    int passedFolds;

    int localEligibleFolds;
    int localPositiveFolds;
    double sumLocalImprovement;

    int structuralEligibleFolds;
    int structuralPositiveFolds;
    double sumStructuralImprovement;

    SampleStat pooledPattern;
    SampleStat pooledParent;

    int lastSelectedFold;
    bool lastFoldEligible;
    bool lastFoldPass;
};

CandidateDef g_candidates[];
FoldSelection g_selections[];
PatternSummary g_summaries[];

SampleStat g_trainStats[];

```

```

SampleStat g_oosStats[];

//--- Быстрые индексы всех категорий
int g_ageCandidate[ZONE_COUNT][AGE_COUNT];
int g_touchCandidate[ZONE_COUNT][AGE_COUNT][TOUCH_BUCKET_COUNT];
int g_depthCandidate[ZONE_COUNT][AGE_COUNT][DEPTH_BUCKET_COUNT];
int g_touchDepthCandidate[ZONE_COUNT][AGE_COUNT][TOUCH_BUCKET_COUNT][DEPTH_BUCKET_COUNT];
int g_approachCandidate[ZONE_COUNT][AGE_COUNT][APPROACH_COUNT];
int g_heightCandidate[ZONE_COUNT][AGE_COUNT][HEIGHT_COUNT];
int g_sessionCandidate[ZONE_COUNT][AGE_COUNT][SESSION_COUNT];

//--- Диапазон и сегменты
datetime g_firstTouchTime = 0;
datetime g_lastTouchTime = 0;
datetime g_segmentStart[];
datetime g_segmentEnd[];
int g_foldCount = 0;

//--- Индексы touches.csv
int t_idxSymbol = -1;
int t_idxTimeframe = -1;
int t_idxZoneType = -1;
int t_idxTouchNumber = -1;
int t_idxTouchStatus = -1;
int t_idxEntryDirection = -1;
int t_idxDecisionTime = -1;
int t_idxAgeTradingHours = -1;
int t_idxZoneHeightATR = -1;
int t_idxDepthPercent = -1;
int t_idxApproachNetATR5 = -1;
int t_idxLocalFirstResult = -1;
int t_idxPrimaryReversalTime = -1;
int t_idxLocalBreakoutTime = -1;
int t_idxStructuralFirstResult = -1;
int t_idxStructuralReversalTime = -1;
int t_idxStructuralBreakoutTime = -1;
int t_idxMFE_CloseATR = -1;
int t_idxMAE_CloseATR = -1;
int t_idxLocalObservationEndTime = -1;

int g_rowsFirstPass = 0;
int g_rowsSecondPass = 0;
int g_rowsIncluded = 0;
int g_rowsSkipped = 0;

```

```
//+-----
```

```
//| Имя текущего таймфрейма
```

```

//+-----
string TimeframeToString(int timeframe)
{
    switch(timeframe)
    {
        case PERIOD_M1: return("M1");
        case PERIOD_M5: return("M5");
        case PERIOD_M15: return("M15");
        case PERIOD_M30: return("M30");
        case PERIOD_H1: return("H1");
        case PERIOD_H4: return("H4");
        case PERIOD_D1: return("D1");
        case PERIOD_W1: return("W1");
        case PERIOD_MN1: return("MN1");
    }

    return(IntegerToString(timeframe));
}

//+-----

//| Удаление BOM

//+-----
string StripBOM(string value)
{
    if(StringLen(value) > 0 && StringGetCharacter(value, 0) ==
65279)
        return(StringSubstr(value, 1));

    if(StringLen(value) >= 3 &&
        StringGetCharacter(value, 0) == 239 &&
        StringGetCharacter(value, 1) == 187 &&
        StringGetCharacter(value, 2) == 191)
    {
        return(StringSubstr(value, 3));
    }

    return(value);
}

//+-----

//| Поиск колонки

//+-----
int FindColumn(string &headers[], string columnName)
{
    int count = ArraySize(headers);

```

```

for(int i = 0; i < count; i++)
{
    string current = headers[i];

    if(i == 0)
        current = StripBOM(current);

    if(current == columnName)
        return(i);
}

return(-1);
}

//+-----

//| Чтение CSV-записи

//+-----
bool ReadCsvRecord(int handle, int columns, string &fields[])
{
    ArrayResize(fields, columns);

    if(FileIsEnding(handle))
        return(false);

    for(int i = 0; i < columns; i++)
    {
        if(FileIsEnding(handle) && i == 0)
            return(false);

        fields[i] = FileReadString(handle);
    }

    return(StringLen(fields[0]) > 0);
}

//+-----

//| Преобразования

//+-----
int ParseInt(string value, int defaultValue = 0)
{
    if(StringLen(value) == 0)
        return(defaultValue);

    return((int)StringToInteger(value));
}

```

```

double ParseDouble(string value, double defaultValue = 0.0)
{
    if(StringLen(value) == 0)
        return(defaultValue);

    return(StringToDouble(value));
}

datetime ParseTime(string value)
{
    if(StringLen(value) == 0)
        return(0);

    return(StringToTime(value));
}

//+-----+
//| Kapta touches.csv
//+-----+
bool MapTouchesColumns(string &headers[])
{
    t_idxSymbol = FindColumn(headers, "Symbol"
);
    t_idxTimeframe = FindColumn(headers,
"Timeframe");
    t_idxZoneType = FindColumn(headers,
"ZoneType");
    t_idxTouchNumber = FindColumn(headers,
"TouchNumber");
    t_idxTouchStatus = FindColumn(headers,
"TouchStatus");
    t_idxEntryDirection = FindColumn(headers,
"EntryDirection");
    t_idxDecisionTime = FindColumn(headers,
"TouchDecisionTime");
    t_idxAgeTradingHours = FindColumn(headers,
"ZoneAgeTradingHours");
    t_idxZoneHeightATR = FindColumn(headers,
"ZoneHeightATR");
    t_idxDepthPercent = FindColumn(headers,
"TouchDepthPercent");
    t_idxApproachNetATR5 = FindColumn(headers,
"ApproachNetMoveATR_5");
    t_idxLocalFirstResult = FindColumn(headers,
"LocalFirstResult");
    t_idxPrimaryReversalTime = FindColumn(headers,
"PrimaryReversalTime");
    t_idxLocalBreakoutTime = FindColumn(headers,
"LocalBreakoutTime");

```

```

    t_idxStructuralFirstResult    = FindColumn(headers,
"StructuralFirstResult");
    t_idxStructuralReversalTime   = FindColumn(headers,
"StructuralReversalTime");
    t_idxStructuralBreakoutTime   = FindColumn(headers,
"StructuralBreakoutTime");
    t_idxMFE_CloseATR             = FindColumn(headers,
"MFE_CloseATR");
    t_idxMAE_CloseATR             = FindColumn(headers,
"MAE_CloseATR");
    t_idxLocalObservationEndTime = FindColumn(headers,
"LocalObservationEndTime");

    if(t_idxSymbol < 0 || t_idxTimeframe < 0 || t_idxZoneType <
0 ||
        t_idxTouchNumber < 0 || t_idxTouchStatus < 0 ||
        t_idxEntryDirection < 0 || t_idxDecisionTime < 0 ||
        t_idxAgeTradingHours < 0 || t_idxZoneHeightATR < 0 ||
        t_idxDepthPercent < 0 || t_idxApproachNetATR5 < 0 ||
        t_idxLocalFirstResult < 0 || t_idxPrimaryReversalTime < 0
    ||
        t_idxLocalBreakoutTime < 0 || t_idxStructuralFirstResult
< 0 ||
        t_idxStructuralReversalTime < 0 ||
        t_idxStructuralBreakoutTime < 0 || t_idxMFE_CloseATR < 0
    ||
        t_idxMAE_CloseATR < 0 || t_idxLocalObservationEndTime < 0
    )
    {
        Print(
"Ошибка: в touches.csv отсутствуют обязательные колонки блока 06."
);
        return(false);
    }

    return(true);
}

```

//+-----

//| Допустимая строка касания

//+-----

```

bool IsIncludedTouch(string &fields[])
{
    if(fields[t_idxTouchStatus] != "TOUCH")
        return(false);

    if(fields[t_idxEntryDirection] != "EXPECTED_SIDE")
        return(false);

    if(FilterCurrentChartOnly)

```

```

{
    if(fields[t_idxSymbol] != Symbol())
        return(false);

    if(fields[t_idxTimeframe] != TimeframeToString(Period()))
        return(false);
}

if(fields[t_idxZoneType] != "BULL" &&
    fields[t_idxZoneType] != "BEAR")
{
    return(false);
}

return(true);
}

```

//+-----

//| Первый проход: диапазон правильных касаний

//+-----

```

bool FindTouchRange()
{
    ResetLastError();

    int handle = FileOpen(
        InputTouchesFileName,
        FILE_READ | FILE_CSV | FILE_ANSI,
        ';'
    );

    if(handle == INVALID_HANDLE)
    {
        Print("Ошибка открытия ", InputTouchesFileName,
            ". Код ошибки: ", GetLastError());
        ResetLastError();
        return(false);
    }

    string headers[];
    ArrayResize(headers, TOUCHES_CSV_COLUMNS);

    for(int i = 0; i < TOUCHES_CSV_COLUMNS; i++)
        headers[i] = FileReadString(handle);

    if(!MapTouchesColumns(headers))
    {
        FileClose(handle);
        return(false);
    }
}

```

```

g_firstTouchTime = 0;
g_lastTouchTime  = 0;
g_rowsFirstPass  = 0;

string fields[];

while(ReadCsvRecord(handle, TOUCHES_CSV_COLUMNS, fields))
{
    g_rowsFirstPass++;

    if(!IsIncludedTouch(fields))
        continue;

    datetime eventTime = ParseTime(fields[t_idxDecisionTime
]);

    if(eventTime <= 0)
        continue;

    if(g_firstTouchTime == 0 || eventTime < g_firstTouchTime)
        g_firstTouchTime = eventTime;

    if(g_lastTouchTime == 0 || eventTime > g_lastTouchTime)
        g_lastTouchTime = eventTime;

    if(ProgressEveryRows > 0 &&
        g_rowsFirstPass % ProgressEveryRows == 0)
    {
        Print("Блок 06, проход 1: прочитано строк ",
g_rowsFirstPass);
    }
}

FileClose(handle);

if(g_firstTouchTime <= 0 || g_lastTouchTime <
g_firstTouchTime)
{
    Print("Ошибка: диапазон правильных касаний не найден.");
    return(false);
}

Print("Блок 06: диапазон касаний: ",
    TimeToString(g_firstTouchTime, TIME_DATE |
TIME_MINUTES),
    " — ",
    TimeToString(g_lastTouchTime, TIME_DATE | TIME_MINUTES
));

return(true);
}

```

```

//+-----
//| Построение равных календарных сегментов
//+-----
bool BuildSegments()
{
    long totalSeconds = (long) (g_lastTouchTime -
g_firstTouchTime) + 1;

    if(totalSeconds < NumberOfSegments)
    {
        Print(
"Ошибка: диапазон истории слишком мал для выбранного числа сегментов
");
        return(false);
    }

    if(ArrayResize(g_segmentStart, NumberOfSegments) !=
NumberOfSegments ||
        ArrayResize(g_segmentEnd, NumberOfSegments) !=
NumberOfSegments)
    {
        Print("Ошибка памяти при создании сегментов.");
        return(false);
    }

    for(int segment = 0; segment < NumberOfSegments; segment++)
    {
        long startOffset = (totalSeconds * segment) /
NumberOfSegments;
        long endOffset =
            (totalSeconds * (segment + 1)) / NumberOfSegments - 1;

        g_segmentStart[segment] =
            (datetime) ((long)g_firstTouchTime + startOffset);

        g_segmentEnd[segment] =
            (datetime) ((long)g_firstTouchTime + endOffset);
    }

    g_segmentEnd[NumberOfSegments - 1] = g_lastTouchTime;
    g_foldCount = NumberOfSegments - TrainingSegments;

    for(int s = 0; s < NumberOfSegments; s++)
    {
        Print("Сегмент ", s + 1, ": ",
            TimeToString(g_segmentStart[s], TIME_DATE |
TIME_MINUTES),
            " - ",
            TimeToString(g_segmentEnd[s], TIME_DATE |
TIME_MINUTES));
    }
}

```

```

    }

    return(true);
}

//+-----
//| Категории блока 03
//+-----
int GetAgeBucket(double ageMinutes)
{
    if(ageMinutes < 15.0)    return(0);
    if(ageMinutes < 30.0)    return(1);
    if(ageMinutes < 60.0)    return(2);
    if(ageMinutes < 120.0)   return(3);
    if(ageMinutes < 240.0)   return(4);
    if(ageMinutes < 480.0)   return(5);
    if(ageMinutes < 1440.0)  return(6);
    if(ageMinutes < 2880.0)  return(7);
    return(8);
}

string AgeBucketName(int bucket)
{
    switch(bucket)
    {
        case 0: return("00_0_15_MIN");
        case 1: return("01_15_30_MIN");
        case 2: return("02_30_60_MIN");
        case 3: return("03_1_2_HOURS");
        case 4: return("04_2_4_HOURS");
        case 5: return("05_4_8_HOURS");
        case 6: return("06_8_24_HOURS");
        case 7: return("07_24_48_HOURS");
        case 8: return("08_48_HOURS_PLUS");
    }

    return("UNKNOWN");
}

int GetTouchBucket(int touchNumber)
{
    if(touchNumber <= 1) return(0);
    if(touchNumber == 2) return(1);
    if(touchNumber == 3) return(2);
    if(touchNumber == 4) return(3);
    return(4);
}

string TouchBucketName(int bucket)

```

```

{
    switch(bucket)
    {
        case 0: return("TOUCH_1");
        case 1: return("TOUCH_2");
        case 2: return("TOUCH_3");
        case 3: return("TOUCH_4");
        case 4: return("TOUCH_5_PLUS");
    }

    return("ALL");
}

int GetDepthBucket(double depthPercent)
{
    if(depthPercent < 25.0) return(0);
    if(depthPercent < 50.0) return(1);
    if(depthPercent < 75.0) return(2);
    return(3);
}

string DepthBucketName(int bucket)
{
    switch(bucket)
    {
        case 0: return("DEPTH_0_25_PCT");
        case 1: return("DEPTH_25_50_PCT");
        case 2: return("DEPTH_50_75_PCT");
        case 3: return("DEPTH_75_100_PCT");
    }

    return("ALL");
}

int GetApproachBucket(double approachNetATR5)
{
    if(approachNetATR5 <= 0.0) return(0);
    if(approachNetATR5 < 0.5) return(1);
    if(approachNetATR5 < 1.0) return(2);
    return(3);
}

string ApproachBucketName(int bucket)
{
    switch(bucket)
    {
        case 0: return("APPROACH_FLAT_OR_AWAY");
        case 1: return("APPROACH_SLOW_LT_0_5_ATR");
        case 2: return("APPROACH_MEDIUM_0_5_1_ATR");
        case 3: return("APPROACH_FAST_GE_1_ATR");
    }
}

```

```

    return("ALL");
}

int GetHeightBucket(double zoneHeightATR)
{
    if(zoneHeightATR < 0.25) return(0);
    if(zoneHeightATR < 0.50) return(1);
    if(zoneHeightATR < 0.75) return(2);
    if(zoneHeightATR < 1.00) return(3);
    return(4);
}

string HeightBucketName(int bucket)
{
    switch(bucket)
    {
        case 0: return("HEIGHT_LT_0_25_ATR");
        case 1: return("HEIGHT_0_25_0_50_ATR");
        case 2: return("HEIGHT_0_50_0_75_ATR");
        case 3: return("HEIGHT_0_75_1_00_ATR");
        case 4: return("HEIGHT_GE_1_00_ATR");
    }

    return("ALL");
}

int GetSessionBucket(datetime decisionTime)
{
    int hour = TimeHour(decisionTime);

    if(hour < AsiaEndHour)
        return(0);

    if(hour < LondonEndHour)
        return(1);

    if(hour < NewYorkEndHour)
        return(2);

    return(3);
}

string SessionBucketName(int bucket)
{
    switch(bucket)
    {
        case 0: return("ASIA_SERVER_TIME");
        case 1: return("LONDON_SERVER_TIME");
        case 2: return("NEW_YORK_SERVER_TIME");
        case 3: return("OTHER_SERVER_TIME");
    }
}

```

```

    return("ALL");
}

string ZoneTypeName(int zoneIndex)
{
    if(zoneIndex == 0)
        return("BULL");

    return("BEAR");
}

int ZoneTypeIndex(string zoneType)
{
    if(zoneType == "BULL")
        return(0);

    if(zoneType == "BEAR")
        return(1);

    return(-1);
}

//+-----

//| Формирование PatternID

//+-----
string BuildPatternID(CandidateDef &candidate)
{
    string touchBucket =
        candidate.touchIndex >= 0
        ? TouchBucketName(candidate.touchIndex)
        : "ALL";

    string depthBucket =
        candidate.depthIndex >= 0
        ? DepthBucketName(candidate.depthIndex)
        : "ALL";

    string approachBucket =
        candidate.approachIndex >= 0
        ? ApproachBucketName(candidate.approachIndex)
        : "ALL";

    string heightBucket =
        candidate.heightIndex >= 0
        ? HeightBucketName(candidate.heightIndex)
        : "ALL";

    string sessionBucket =
        candidate.sessionIndex >= 0

```

```

        ? SessionBucketName(candidate.sessionIndex)
        : "ALL";

return(
    candidate.patternType + "_TRADING_TIME_" +
    Symbol() + "_" + TimeframeToString(Period()) + "_" +
    ZoneTypeName(candidate.zoneIndex) + "_" +
    AgeBucketName(candidate.ageIndex) + "_" +
    touchBucket + "_" + depthBucket + "_" +
    approachBucket + "_" + heightBucket + "_" +
    sessionBucket
);
}

```

//+-----

/// | Добавление кандидата

//+-----

```

int AddCandidate(
    string patternType,
    int zoneIndex,
    int ageIndex,
    int touchIndex,
    int depthIndex,
    int approachIndex,
    int heightIndex,
    int sessionIndex
)
{
    int size = ArraySize(g_candidates);

    if(ArrayResize(g_candidates, size + 1) != size + 1)
        return(-1);

    CandidateDef candidate;
    candidate.patternType = patternType;
    candidate.zoneIndex = zoneIndex;
    candidate.ageIndex = ageIndex;
    candidate.touchIndex = touchIndex;
    candidate.depthIndex = depthIndex;
    candidate.approachIndex = approachIndex;
    candidate.heightIndex = heightIndex;
    candidate.sessionIndex = sessionIndex;
    candidate.patternID = "";
    candidate.patternID = BuildPatternID(candidate);

    g_candidates[size] = candidate;
    return(size);
}

```

```

//+-----
//| Создание полного пространства кандидатов
//+-----
bool BuildCandidates()
{
    ArrayResize(g_candidates, 0);

    for(int z0 = 0; z0 < ZONE_COUNT; z0++)
    {
        for(int a0 = 0; a0 < AGE_COUNT; a0++)
        {
            g_ageCandidate[z0][a0] = -1;

            for(int t0 = 0; t0 < TOUCH_BUCKET_COUNT; t0++)
            {
                g_touchCandidate[z0][a0][t0] = -1;

                for(int d0 = 0; d0 < DEPTH_BUCKET_COUNT; d0++)
                    g_touchDepthCandidate[z0][a0][t0][d0] = -1;
            }

            for(int d1 = 0; d1 < DEPTH_BUCKET_COUNT; d1++)
                g_depthCandidate[z0][a0][d1] = -1;

            for(int ap0 = 0; ap0 < APPROACH_COUNT; ap0++)
                g_approachCandidate[z0][a0][ap0] = -1;

            for(int h0 = 0; h0 < HEIGHT_COUNT; h0++)
                g_heightCandidate[z0][a0][h0] = -1;

            for(int s0 = 0; s0 < SESSION_COUNT; s0++)
                g_sessionCandidate[z0][a0][s0] = -1;
        }
    }

    for(int z = 0; z < ZONE_COUNT; z++)
    {
        for(int age = 0; age < AGE_COUNT; age++)
        {
            int index = AddCandidate(
                "AGE_ONLY", z, age, -1, -1, -1, -1, -1
            );

            if(index < 0)
                return(false);

            g_ageCandidate[z][age] = index;
        }
    }
}

```

```

    for(int touch = 0; touch < TOUCH_BUCKET_COUNT; touch
++)
    {
        index = AddCandidate(
            "AGE_TOUCH", z, age, touch, -1, -1, -1, -1
        );

        if(index < 0)
            return(false);

        g_touchCandidate[z][age][touch] = index;
    }

    for(int depth = 0; depth < DEPTH_BUCKET_COUNT; depth
++)
    {
        index = AddCandidate(
            "AGE_DEPTH", z, age, -1, depth, -1, -1, -1
        );

        if(index < 0)
            return(false);

        g_depthCandidate[z][age][depth] = index;
    }

    for(int touch2 = 0; touch2 < TOUCH_BUCKET_COUNT;
touch2++)
    {
        for(int depth2 = 0; depth2 < DEPTH_BUCKET_COUNT;
depth2++)
        {
            index = AddCandidate(
                "AGE_TOUCH_DEPTH",
                z,
                age,
                touch2,
                depth2,
                -1,
                -1,
                -1
            );

            if(index < 0)
                return(false);

            g_touchDepthCandidate[z][age][touch2][depth2] =
index;
        }
    }
}

```

```

        for(int approach = 0; approach < APPROACH_COUNT;
approach++)
        {
            index = AddCandidate(
                "AGE_APPROACH", z, age, -1, -1, approach, -1, -1
            );

            if(index < 0)
                return(false);

            g_approachCandidate[z][age][approach] = index;
        }

        for(int height = 0; height < HEIGHT_COUNT; height++)
        {
            index = AddCandidate(
                "AGE_HEIGHT", z, age, -1, -1, -1, height, -1
            );

            if(index < 0)
                return(false);

            g_heightCandidate[z][age][height] = index;
        }

        for(int session = 0; session < SESSION_COUNT; session
++)
        {
            index = AddCandidate(
                "AGE_SESSION", z, age, -1, -1, -1, -1, session
            );

            if(index < 0)
                return(false);

            g_sessionCandidate[z][age][session] = index;
        }
    }

    Print("Блок 06: создано кандидатов: ", ArraySize(
g_candidates));
    return(true);
}

//+-----

//| Работа со статистикой

//+-----
void ResetStat(SampleStat &stat)

```

```

{
    stat.samples = 0;
    stat.localReversal = 0;
    stat.localBreakout = 0;
    stat.structuralOpposite = 0;
    stat.structuralSameType = 0;
    stat.sumMFE = 0.0;
    stat.countMFE = 0;
    stat.sumMAE = 0.0;
    stat.countMAE = 0;
}

void AddStat(SampleStat &target, SampleStat &source)
{
    target.samples += source.samples;
    target.localReversal += source.localReversal;
    target.localBreakout += source.localBreakout;
    target.structuralOpposite += source.structuralOpposite;
    target.structuralSameType += source.structuralSameType;
    target.sumMFE += source.sumMFE;
    target.countMFE += source.countMFE;
    target.sumMAE += source.sumMAE;
    target.countMAE += source.countMAE;
}

bool AllocateStatistics()
{
    int candidateCount = ArraySize(g_candidates);
    int total = candidateCount * g_foldCount;

    if(ArrayResize(g_trainStats, total) != total ||
        ArrayResize(g_oosStats, total) != total)
    {
        Print("Ошибка памяти при создании массивов статистики.");
        return(false);
    }

    for(int i = 0; i < total; i++)
    {
        ResetStat(g_trainStats[i]);
        ResetStat(g_oosStats[i]);
    }

    return(true);
}

int FoldStatIndex(int foldIndex, int candidateIndex)
{
    return(foldIndex * ArraySize(g_candidates) + candidateIndex);
}

```

```

//+-----
//| Добавление одного касания с цензурированием будущих результатов
//+-----
void UpdateOneStat(
    SampleStat &stat,
    string &fields[],
    datetime cutoffTime
)
{
    stat.samples++;

    string localResult = fields[t_idxLocalFirstResult];

    if(localResult == "REVERSAL_FIRST")
    {
        datetime resultTime = ParseTime(fields[
t_idxPrimaryReversalTime]);

        if(resultTime > 0 && resultTime <= cutoffTime)
            stat.localReversal++;
    }
    else if(localResult == "BREAKOUT_FIRST")
    {
        datetime resultTime2 = ParseTime(fields[
t_idxLocalBreakoutTime]);

        if(resultTime2 > 0 && resultTime2 <= cutoffTime)
            stat.localBreakout++;
    }

    string structuralResult = fields[t_idxStructuralFirstResult
];

    if(structuralResult == "OPPOSITE_ZONE_FIRST")
    {
        datetime structuralTime =
            ParseTime(fields[t_idxStructuralReversalTime]);

        if(structuralTime > 0 && structuralTime <= cutoffTime)
            stat.structuralOpposite++;
    }
    else if(structuralResult == "SAME_TYPE_ZONE_FIRST")
    {
        datetime structuralTime2 =
            ParseTime(fields[t_idxStructuralBreakoutTime]);

        if(structuralTime2 > 0 && structuralTime2 <= cutoffTime)
            stat.structuralSameType++;
    }
}

```

```

// MFE/MAE безопасны только при полном завершении локального окна
datetime observationEnd =
    ParseTime(fields[t_idxLocalObservationEndTime]);

if(observationEnd > 0 && observationEnd <= cutoffTime)
{
    if(StringLen(fields[t_idxMFE_CloseATR]) > 0)
    {
        stat.sumMFE += StringToDouble(fields[t_idxMFE_CloseATR
]);
        stat.countMFE++;
    }

    if(StringLen(fields[t_idxMAE_CloseATR]) > 0)
    {
        stat.sumMAE += StringToDouble(fields[t_idxMAE_CloseATR
]);
        stat.countMAE++;
    }
}

//+-----

//| Обновление семи соответствующих групп одного касания

//+-----
void UpdateCandidateSet(
    SampleStat &stats[],
    int foldIndex,
    int zoneIndex,
    int ageIndex,
    int touchIndex,
    int depthIndex,
    int approachIndex,
    int heightIndex,
    int sessionIndex,
    string &fields[],
    datetime cutoffTime
)
{
    int indexes[7];

    indexes[0] = g_ageCandidate[zoneIndex][ageIndex];
    indexes[1] = g_touchCandidate[zoneIndex][ageIndex][
touchIndex];
    indexes[2] = g_depthCandidate[zoneIndex][ageIndex][
depthIndex];
    indexes[3] =

```

```

        g_touchDepthCandidate[zoneIndex][ageIndex][touchIndex][
depthIndex];
        indexes[4] =
            g_approachCandidate[zoneIndex][ageIndex][approachIndex];
        indexes[5] = g_heightCandidate[zoneIndex][ageIndex][
heightIndex];
        indexes[6] = g_sessionCandidate[zoneIndex][ageIndex][
sessionIndex];

        for(int i = 0; i < 7; i++)
        {
            int statIndex = FoldStatIndex(foldIndex, indexes[i]);
            UpdateOneStat(stats[statIndex], fields, cutoffTime);
        }
    }

//+-----
//| Второй проход: обучение и OOS без выхода за границы окон
//+-----
bool AggregateRollingWindows()
{
    ResetLastError();

    int handle = FileOpen(
        InputTouchesFileName,
        FILE_READ | FILE_CSV | FILE_ANSI,
        ';'
    );

    if(handle == INVALID_HANDLE)
    {
        Print("Ошибка повторного открытия ", InputTouchesFileName
,
            ". Код ошибки: ", GetLastError());
        ResetLastError();
        return(false);
    }

    string headers[];
    ArrayResize(headers, TOUCHES_CSV_COLUMNS);

    for(int i = 0; i < TOUCHES_CSV_COLUMNS; i++)
        headers[i] = FileReadString(handle);

    if(!MapTouchesColumns(headers))
    {
        FileClose(handle);
        return(false);
    }
}

```

```

string fields[];
g_rowsSecondPass = 0;
g_rowsIncluded = 0;
g_rowsSkipped = 0;

while(ReadCsvRecord(handle, TOUCHES_CSV_COLUMNS, fields))
{
    g_rowsSecondPass++;

    if(!IsIncludedTouch(fields))
    {
        g_rowsSkipped++;
        continue;
    }

    datetime decisionTime = ParseTime(fields[
t_idxDecisionTime]);
    int touchNumber = ParseInt(fields[t_idxTouchNumber], -1);
    double tradingAgeMinutes =
        ParseDouble(fields[t_idxAgeTradingHours], -1.0) * 60.0
;

    double zoneHeightATR =
        ParseDouble(fields[t_idxZoneHeightATR], -1.0);
    double depthPercent =
        ParseDouble(fields[t_idxDepthPercent], -1.0);
    double approachNetATR5 =
        ParseDouble(fields[t_idxApproachNetATR5], 0.0);
    int zoneIndex = ZoneTypeIndex(fields[t_idxZoneType]);

    if(decisionTime <= 0 || touchNumber < 1 ||
        tradingAgeMinutes < 0.0 || zoneHeightATR < 0.0 ||
        depthPercent < 0.0 || depthPercent > 100.0001 ||
        zoneIndex < 0)
    {
        g_rowsSkipped++;
        continue;
    }

    int ageIndex = GetAgeBucket(tradingAgeMinutes);
    int touchIndex = GetTouchBucket(touchNumber);
    int depthIndex = GetDepthBucket(depthPercent);
    int approachIndex = GetApproachBucket(approachNetATR5);
    int heightIndex = GetHeightBucket(zoneHeightATR);
    int sessionIndex = GetSessionBucket(decisionTime);

    bool used = false;

    for(int fold = 0; fold < g_foldCount; fold++)
    {
        int trainFirstSegment = fold;
        int trainLastSegment = fold + TrainingSegments - 1;
        int oosSegment = fold + TrainingSegments;

```

```

];

datetime trainStart = g_segmentStart[trainFirstSegment];
datetime trainEnd = g_segmentEnd[trainLastSegment];
datetime oosStart = g_segmentStart[oosSegment];
datetime oosEnd = g_segmentEnd[oosSegment];

if(decisionTime >= trainStart && decisionTime <=
trainEnd)
{
    UpdateCandidateSet(
        g_trainStats,
        fold,
        zoneIndex,
        ageIndex,
        touchIndex,
        depthIndex,
        approachIndex,
        heightIndex,
        sessionIndex,
        fields,
        trainEnd
    );

    used = true;
}

if(decisionTime >= oosStart && decisionTime <= oosEnd)
{
    UpdateCandidateSet(
        g_oosStats,
        fold,
        zoneIndex,
        ageIndex,
        touchIndex,
        depthIndex,
        approachIndex,
        heightIndex,
        sessionIndex,
        fields,
        oosEnd
    );

    used = true;
}
}

if(used)
    g_rowsIncluded++;

if(ProgressEveryRows > 0 &&
    g_rowsSecondPass % ProgressEveryRows == 0)
{

```

```

        Print("Блок 06, проход 2: прочитано строк ",
g_rowsSecondPass);
    }
}

FileClose(handle);

Print("Блок 06: строк прочитано: ", g_rowsSecondPass,
      ", использовано: ", g_rowsIncluded,
      ", пропущено: ", g_rowsSkipped);

return(true);
}

//+-----

//| Статистические функции

//+-----
double ClampDouble(double value, double minimum, double maximum)
{
    if(value < minimum)
        return(minimum);

    if(value > maximum)
        return(maximum);

    return(value);
}

double Percent(int successes, int total)
{
    if(total <= 0)
        return(-1.0);

    return(100.0 * (double)successes / (double)total);
}

double WilsonLower95(int successes, int total)
{
    if(total <= 0)
        return(-1.0);

    double z = 1.959963984540054;
    double n = (double)total;
    double p = (double)successes / n;
    double z2 = z * z;

    double numerator =
        p + z2 / (2.0 * n) -

```

```

        z * Math.Sqrt((p * (1.0 - p) + z2 / (4.0 * n)) / n);

double denominator = 1.0 + z2 / n;

return(100.0 * numerator / denominator);
}

double Average(double sum, int count)
{
    if(count <= 0)
        return(-1.0);

    return(sum / (double)count);
}

double SafeRatio(double numerator, double denominator)
{
    if(numerator < 0.0 || denominator <= 0.0)
        return(-1.0);

    return(numerator / denominator);
}

//+-----

//| Сила родителя по обучающему окну

//+-----

double ParentStrength(int foldIndex, int candidateIndex)
{
    if(candidateIndex < 0)
        return(-1.0);

    SampleStat stat =
        g_trainStats[FoldStatIndex(foldIndex, candidateIndex)];

    int localDecisive = stat.localReversal + stat.localBreakout;
    int structuralDecisive =
        stat.structuralOpposite + stat.structuralSameType;

    double localWilson =
        MathMax(WilsonLower95(stat.localReversal, localDecisive),
0.0);

    double structuralWilson =
        MathMax(
            WilsonLower95(stat.structuralOpposite,
structuralDecisive),
0.0
        );
}

```

```

    return(0.5 * localWilson + 0.5 * structuralWilson);
}

//+-----

//| Родитель выбирается только по обучающим данным

//+-----
int FindParentIndex(int foldIndex, CandidateDef &candidate)
{
    int ageParent =
        g_ageCandidate[candidate.zoneIndex][candidate.ageIndex];

    if(candidate.patternType != "AGE_TOUCH_DEPTH")
        return(ageParent);

    int touchParent =
        g_touchCandidate
        [candidate.zoneIndex]
        [candidate.ageIndex]
        [candidate.touchIndex];

    int depthParent =
        g_depthCandidate
        [candidate.zoneIndex]
        [candidate.ageIndex]
        [candidate.depthIndex];

    int bestParent = ageParent;
    double bestStrength = ParentStrength(foldIndex, ageParent);

    double touchStrength = ParentStrength(foldIndex, touchParent
);

    if(touchStrength > bestStrength)
    {
        bestStrength = touchStrength;
        bestParent = touchParent;
    }

    double depthStrength = ParentStrength(foldIndex, depthParent
);

    if(depthStrength > bestStrength)
        bestParent = depthParent;

    return(bestParent);
}

string BuildSelectionReason(
    double localImprovement,

```

```

    double structuralImprovement
)
{
    bool localEdge =
        localImprovement >= MinTrainImprovementPctPoints;

    bool structuralEdge =
        structuralImprovement >= MinTrainImprovementPctPoints;

    if(localEdge && structuralEdge)
        return("LOCAL_AND_STRUCTURAL_EDGE");

    if(localEdge)
        return("LOCAL_EDGE");

    if(structuralEdge)
        return("STRUCTURAL_EDGE");

    return("NO_EDGE");
}

string BuildReliabilityTier(
    SampleStat &stat,
    double localImprovement,
    double structuralImprovement
)
{
    int localDecisive = stat.localReversal + stat.localBreakout;
    int structuralDecisive =
        stat.structuralOpposite + stat.structuralSameType;

    bool bothEdges =
        localImprovement >= MinTrainImprovementPctPoints &&
        structuralImprovement >= MinTrainImprovementPctPoints;

    if(stat.samples >= StrongTrainSamples &&
        localDecisive >= StrongTrainSamples &&
        structuralDecisive >= StrongTrainSamples &&
        bothEdges)
    {
        return("A_STRONG_BOTH");
    }

    if(stat.samples >= StrongTrainSamples)
        return("B_STRONG_SAMPLE");

    return("C_RELIABLE");
}

void CalculateScore(
    SampleStat &stat,
    double localWilson,

```

```

double structuralWilson,
double localImprovement,
double structuralImprovement,
double &sampleFactor,
double &payoffScore,
double &improvementScore,
double &finalScore
)
{
double denominator = MathMax((double)StrongTrainSamples, 1.0
);
double sampleRatio =
    MathMin((double)stat.samples / denominator, 1.0);

sampleFactor = MathSqrt(MathMax(sampleRatio, 0.0));

double avgMFE = Average(stat.sumMFE, stat.countMFE);
double avgMAE = Average(stat.sumMAE, stat.countMAE);
double ratio = SafeRatio(avgMFE, avgMAE);

if(ratio < 0.0)
    payoffScore = 0.0;
else
    payoffScore =
        ClampDouble((ratio - 0.5) / 1.5, 0.0, 1.0) * 100.0;

double positiveImprovement =
    MathMax(localImprovement, 0.0) +
    MathMax(structuralImprovement, 0.0);

improvementScore =
    ClampDouble(positiveImprovement / 20.0, 0.0, 1.0) * 100.0
;

double weightSum =
    WeightLocalWilson +
    WeightStructuralWilson +
    WeightMfeMae +
    WeightImprovement;

if(weightSum <= 0.0)
    weightSum = 1.0;

double rawScore =
    (
        WeightLocalWilson * ClampDouble(localWilson, 0.0,
100.0) +
        WeightStructuralWilson *
            ClampDouble(structuralWilson, 0.0, 100.0) +
        WeightMfeMae * payoffScore +
        WeightImprovement * improvementScore
    ) / weightSum;

```

```

    finalScore = rawScore * (0.75 + 0.25 * sampleFactor);
}

//+-----

//| Добавление предварительно отобранного результата

//+-----
bool AddFoldSelection(
    FoldSelection &array[],
    int foldIndex,
    int candidateIndex,
    int parentIndex,
    string selectionReason,
    string reliabilityTier,
    double trainLocalWilson,
    double trainStructuralWilson,
    double parentLocalWilson,
    double parentStructuralWilson,
    double localImprovement,
    double structuralImprovement,
    double sampleFactor,
    double payoffScore,
    double improvementScore,
    double finalScore
)
{
    int size = ArraySize(array);

    if(ArrayResize(array, size + 1) != size + 1)
        return(false);

    FoldSelection item;
    item.foldIndex = foldIndex;
    item.candidateIndex = candidateIndex;
    item.parentIndex = parentIndex;
    item.foldRank = 0;
    item.selectionReason = selectionReason;
    item.reliabilityTier = reliabilityTier;
    item.trainLocalWilson = trainLocalWilson;
    item.trainStructuralWilson = trainStructuralWilson;
    item.parentLocalWilson = parentLocalWilson;
    item.parentStructuralWilson = parentStructuralWilson;
    item.localImprovement = localImprovement;
    item.structuralImprovement = structuralImprovement;
    item.sampleFactor = sampleFactor;
    item.payoffScore = payoffScore;
    item.improvementScore = improvementScore;
    item.finalScore = finalScore;

    array[size] = item;
}

```

```

    return(true);
}

//+-----

//| Сортировка одного обучающего рейтинга

//+-----
bool ShouldComeBefore(FoldSelection &first, FoldSelection &
second)
{
    if(MathAbs(first.finalScore - second.finalScore) > 0.0000001
)
        return(first.finalScore > second.finalScore);

    SampleStat firstStat =
        g_trainStats[FoldStatIndex(first.foldIndex,
first.candidateIndex)];

    SampleStat secondStat =
        g_trainStats[FoldStatIndex(second.foldIndex,
second.candidateIndex)];

    if(firstStat.samples != secondStat.samples)
        return(firstStat.samples > secondStat.samples);

    if(first.trainStructuralWilson !=
second.trainStructuralWilson)
        return(first.trainStructuralWilson >
second.trainStructuralWilson);

    return(first.trainLocalWilson > second.trainLocalWilson);
}

void SortFoldSelections(FoldSelection &items[])
{
    int count = ArraySize(items);

    for(int i = 0; i < count - 1; i++)
    {
        int best = i;

        for(int j = i + 1; j < count; j++)
        {
            if(ShouldComeBefore(items[j], items[best]))
                best = j;
        }

        if(best != i)
        {
            FoldSelection temp = items[i];

```

```

        items[i] = items[best];
        items[best] = temp;
    }
}

for(int k = 0; k < count; k++)
    items[k].foldRank = k + 1;
}

//+-----

//| Отбор кандидатов отдельно для каждого прошлого окна

//+-----
bool SelectPatterns()
{
    ArrayResize(g_selections, 0);

    int candidateCount = ArraySize(g_candidates);

    for(int fold = 0; fold < g_foldCount; fold++)
    {
        FoldSelection foldItems[];
        ArrayResize(foldItems, 0);

        for(int candidateIndex = 0;
            candidateIndex < candidateCount;
            candidateIndex++)
        {
            CandidateDef candidate = g_candidates[candidateIndex];

            if(candidate.patternType == "AGE_ONLY")
                continue;

            SampleStat stat =
                g_trainStats[FoldStatIndex(fold, candidateIndex)];

            int localDecisive = stat.localReversal +
stat.localBreakout;
            int structuralDecisive =
                stat.structuralOpposite + stat.structuralSameType;

            if(stat.samples < MinTrainSamples ||
                localDecisive < MinTrainLocalDecisive ||
                structuralDecisive < MinTrainStructuralDecisive)
            {
                continue;
            }

            double localWilson =
                WilsonLower95(stat.localReversal, localDecisive);

```

```

double structuralWilson =
    WilsonLower95(stat.structuralOpposite,
structuralDecisive);

if(localWilson < MinTrainLocalWilson &&
    structuralWilson < MinTrainStructuralWilson)
{
    continue;
}

double avgMFE = Average(stat.sumMFE, stat.countMFE);
double avgMAE = Average(stat.sumMAE, stat.countMAE);
double ratio = SafeRatio(avgMFE, avgMAE);

if(MinTrainMfeMaeRatio > 0.0 &&
    (ratio < 0.0 || ratio < MinTrainMfeMaeRatio))
{
    continue;
}

int parentIndex = FindParentIndex(fold, candidate);

if(parentIndex < 0)
    continue;

SampleStat parentStat =
    g_trainStats[FoldStatIndex(fold, parentIndex)];

int parentLocalDecisive =
    parentStat.localReversal + parentStat.localBreakout
;

int parentStructuralDecisive =
    parentStat.structuralOpposite +
    parentStat.structuralSameType;

double parentLocalWilson =
    WilsonLower95(
        parentStat.localReversal,
        parentLocalDecisive
    );

double parentStructuralWilson =
    WilsonLower95(
        parentStat.structuralOpposite,
        parentStructuralDecisive
    );

if(parentLocalWilson < 0.0 || parentStructuralWilson <
0.0)
    continue;

```

```

double localImprovement =
    localWilson - parentLocalWilson;

double structuralImprovement =
    structuralWilson - parentStructuralWilson;

if(localImprovement < MinTrainImprovementPctPoints &&
    structuralImprovement <
MinTrainImprovementPctPoints)
{
    continue;
}

double sampleFactor;
double payoffScore;
double improvementScore;
double finalScore;

CalculateScore(
    stat,
    localWilson,
    structuralWilson,
    localImprovement,
    structuralImprovement,
    sampleFactor,
    payoffScore,
    improvementScore,
    finalScore
);

if(!AddFoldSelection(
    foldItems,
    fold,
    candidateIndex,
    parentIndex,
    BuildSelectionReason(
        localImprovement,
        structuralImprovement
    ),
    BuildReliabilityTier(
        stat,
        localImprovement,
        structuralImprovement
    ),
    localWilson,
    structuralWilson,
    parentLocalWilson,
    parentStructuralWilson,
    localImprovement,
    structuralImprovement,
    sampleFactor,
    payoffScore,

```

```

        improvementScore,
        finalScore
    ))
    {
        Print("Ошибка памяти при отборе паттернов.");
        return(false);
    }
}

SortFoldSelections(foldItems);

int saveCount = ArraySize(foldItems);

if(MaxSelectedPatternsPerFold > 0 &&
    saveCount > MaxSelectedPatternsPerFold)
{
    saveCount = MaxSelectedPatternsPerFold;
}

for(int item = 0; item < saveCount; item++)
{
    int globalSize = ArraySize(g_selections);

    if(ArrayResize(g_selections, globalSize + 1) !=
globalSize + 1)
    {
        Print("Ошибка памяти при сохранении отбора.");
        return(false);
    }

    g_selections[globalSize] = foldItems[item];
}

Print("Блок 06, окно ", fold + 1,
      ": найдено паттернов на обучении ", ArraySize(
foldItems),
      ", сохранено ", saveCount);
}

return(true);
}

//+-----

//| Требуемые типы преимущества

//+-----
bool IsLocalEdgeRequired(string selectionReason)
{
    return(
        selectionReason == "LOCAL_EDGE" ||

```

```

        selectionReason == "LOCAL_AND_STRUCTURAL_EDGE"
    );
}

bool IsStructuralEdgeRequired(string selectionReason)
{
    return(
        selectionReason == "STRUCTURAL_EDGE" ||
        selectionReason == "LOCAL_AND_STRUCTURAL_EDGE"
    );
}

//+-----
//| Расчёт OOS-состояния выбранного паттерна
//+-----
void CalculateOosState(
    FoldSelection &selection,
    bool &eligibleLocal,
    bool &eligibleStructural,
    double &localPct,
    double &parentLocalPct,
    double &localImprovement,
    double &structuralPct,
    double &parentStructuralPct,
    double &structuralImprovement,
    bool &localEdge,
    bool &structuralEdge,
    bool &foldEligible,
    bool &foldPass
)
{
    SampleStat stat =
        g_oosStats[
            FoldStatIndex(selection.foldIndex,
selection.candidateIndex)
        ];

    SampleStat parentStat =
        g_oosStats[
            FoldStatIndex(selection.foldIndex,
selection.parentIndex)
        ];

    int localDecisive = stat.localReversal + stat.localBreakout;
    int parentLocalDecisive =
        parentStat.localReversal + parentStat.localBreakout;

    int structuralDecisive =
        stat.structuralOpposite + stat.structuralSameType;

```

```

    int parentStructuralDecisive =
        parentStat.structuralOpposite +
parentStat.structuralSameType;

    localPct = Percent(stat.localReversal, localDecisive);
    parentLocalPct =
        Percent(parentStat.localReversal, parentLocalDecisive);

    structuralPct =
        Percent(stat.structuralOpposite, structuralDecisive);

    parentStructuralPct =
        Percent(parentStat.structuralOpposite,
parentStructuralDecisive);

    localImprovement = -1.0;
    structuralImprovement = -1.0;

    if(localPct >= 0.0 && parentLocalPct >= 0.0)
        localImprovement = localPct - parentLocalPct;

    if(structuralPct >= 0.0 && parentStructuralPct >= 0.0)
        structuralImprovement = structuralPct -
parentStructuralPct;

    eligibleLocal =
        stat.samples >= MinOosSamples &&
        localDecisive >= MinOosLocalDecisive &&
        parentLocalDecisive >= MinOosLocalDecisive;

    eligibleStructural =
        stat.samples >= MinOosSamples &&
        structuralDecisive >= MinOosStructuralDecisive &&
        parentStructuralDecisive >= MinOosStructuralDecisive;

    localEdge =
        eligibleLocal &&
        localImprovement >= MinOosImprovementPctPoints;

    structuralEdge =
        eligibleStructural &&
        structuralImprovement >= MinOosImprovementPctPoints;

    foldEligible = true;
    foldPass = true;

    if(IsLocalEdgeRequired(selection.selectionReason))
    {
        foldEligible = foldEligible && eligibleLocal;
        foldPass = foldPass && localEdge;
    }

```

```

    if(IsStructuralEdgeRequired(selection.selectionReason))
    {
        foldEligible = foldEligible && eligibleStructural;
        foldPass = foldPass && structuralEdge;
    }

    foldPass = foldEligible && foldPass;
}

//+-----

//| CSV-утилиты

//+-----
string EscapeCsv(string value)
{
    bool needQuotes =
        StringFind(value, ";") >= 0 ||
        StringFind(value, "\"") >= 0 ||
        StringFind(value, "\r") >= 0 ||
        StringFind(value, "\n") >= 0;

    if(!needQuotes)
        return(value);

    StringReplace(value, "\"", "\\\"");
    return("\"" + value + "\"");
}

void AppendField(string &line, string value)
{
    if(StringLen(line) > 0)
        line += ";";

    line += EscapeCsv(value);
}

string DoubleOrBlank(double value, int digits)
{
    if(value < 0.0 || value == EMPTY_VALUE)
        return("");

    return(DoubleToString(value, digits));
}

string SignedDoubleOrBlank(double value, int digits, bool valid)
{
    if(!valid)
        return("");
}

```

```

    return(DoubleToString(value, digits));
}

string YesNo(bool value)
{
    return(value ? "YES" : "NO");
}

string DateTimeText(datetime value)
{
    if(value <= 0)
        return("");

    return(TimeToString(value, TIME_DATE | TIME_MINUTES));
}

//+-----

//| Детальный файл: один выбранный паттерн в одном OOS-окне

//+-----
bool SaveWalkForwardDetails()
{
    ResetLastError();

    int handle = FileOpen(
        OutputWalkForwardFileName,
        FILE_WRITE | FILE_TXT | FILE_ANSI
    );

    if(handle == INVALID_HANDLE)
    {
        Print("Ошибка создания ", OutputWalkForwardFileName,
            ". Код ошибки: ", GetLastError());
        ResetLastError();
        return(false);
    }

    string header =

    "Fold;TrainSegmentFirst;TrainSegmentLast;TrainStart;TrainEnd;" +
    "OOSSegment;OOSStart;OOSEnd;FoldRank;PatternID;ParentPatternID;" +
    "SelectionReason;ReliabilityTier;PatternType;AgeBasis;Symbol;" +
    "Timeframe;ZoneType;AgeBucket;TouchBucket;DepthBucket;" +
    "ApproachBucket;ZoneHeightBucket;SessionBucket;TrainSamples;" +
    "TrainLocalDecisive;TrainLocalReversalPct;"

```

```

    "TrainParentLocalDecisive;TrainParentLocalReversalPct;" +
    "TrainParentLocalWilsonLower95;TrainLocalImprovementPctPoints;"
    +
    "TrainStructuralDecisive;TrainStructuralOpposite;" +
    "TrainStructuralOppositePct;TrainStructuralWilsonLower95;" +
    "TrainParentStructuralDecisive;TrainParentStructuralOppositePct;"
    +
    "TrainParentStructuralWilsonLower95;" +
    "TrainStructuralImprovementPctPoints;TrainAvgMFE_CloseATR;" +
    "TrainAvgMAE_CloseATR;TrainMFE_MAE_Ratio;TrainFinalScore;" +
    "OOSamples;OOSLocalDecisive;OOSLocalReversals;" +
    "OOSLocalReversalPct;OOSLocalWilsonLower95;OOSParentSamples;" +
    "OOSParentLocalDecisive;OOSParentLocalReversalPct;" +
    "OOSParentLocalWilsonLower95;OOSLocalImprovementPctPoints;" +
    "OOSStructuralDecisive;OOSStructuralOpposite;" +
    "OOSStructuralOppositePct;OOSStructuralWilsonLower95;" +
    "OOSParentStructuralDecisive;OOSParentStructuralOppositePct;" +
    "OOSParentStructuralWilsonLower95;" +
    "OOSStructuralImprovementPctPoints;OOSAvgMFE_CloseATR;" +
    "OOSAvgMAE_CloseATR;OOSMFE_MAE_Ratio;EligibleLocal;" +
    "EligibleStructural;LocalEdge;StructuralEdge;OOSEligible;" +
    "OOSPass\r\n";

```

```

FileWriteString(handle, header);

```

```

int passCount = 0;

```

```

int eligibleCount = 0;

```

```

for(int i = 0; i < ArraySize(g_selections); i++)
{
    FoldSelection selection = g_selections[i];
    CandidateDef candidate = g_candidates[
selection.candidateIndex];
    CandidateDef parent = g_candidates[selection.parentIndex
];

    SampleStat trainStat =
        g_trainStats[
            FoldStatIndex(selection.foldIndex,
selection.candidateIndex)
        ];

    SampleStat trainParent =
        g_trainStats[

```

```

        FoldStatIndex(selection.foldIndex,
selection.parentIndex)
    ];

    SampleStat oosStat =
        g_oosStats[
            FoldStatIndex(selection.foldIndex,
selection.candidateIndex)
        ];

    SampleStat oosParent =
        g_oosStats[
            FoldStatIndex(selection.foldIndex,
selection.parentIndex)
        ];

    int trainLocalDecisive =
        trainStat.localReversal + trainStat.localBreakout;
    int trainParentLocalDecisive =
        trainParent.localReversal + trainParent.localBreakout;
    int trainStructuralDecisive =
        trainStat.structuralOpposite +
trainStat.structuralSameType;
    int trainParentStructuralDecisive =
        trainParent.structuralOpposite +
trainParent.structuralSameType;

    double trainLocalPct =
        Percent(trainStat.localReversal, trainLocalDecisive);
    double trainParentLocalPct =
        Percent(trainParent.localReversal,
trainParentLocalDecisive);
    double trainStructuralPct =
        Percent(trainStat.structuralOpposite,
trainStructuralDecisive);
    double trainParentStructuralPct =
        Percent(
            trainParent.structuralOpposite,
            trainParentStructuralDecisive
        );

    double trainAvgMFE =
        Average(trainStat.sumMFE, trainStat.countMFE);
    double trainAvgMAE =
        Average(trainStat.sumMAE, trainStat.countMAE);
    double trainRatio = SafeRatio(trainAvgMFE, trainAvgMAE);

    int oosLocalDecisive =
        oosStat.localReversal + oosStat.localBreakout;
    int oosParentLocalDecisive =
        oosParent.localReversal + oosParent.localBreakout;
    int oosStructuralDecisive =

```

```

        oosStat.structuralOpposite +
oosStat.structuralSameType;
    int oosParentStructuralDecisive =
        oosParent.structuralOpposite +
oosParent.structuralSameType;

    bool eligibleLocal;
    bool eligibleStructural;
    double oosLocalPct;
    double oosParentLocalPct;
    double oosLocalImprovement;
    double oosStructuralPct;
    double oosParentStructuralPct;
    double oosStructuralImprovement;
    bool localEdge;
    bool structuralEdge;
    bool foldEligible;
    bool foldPass;

    CalculateOosState(
        selection,
        eligibleLocal,
        eligibleStructural,
        oosLocalPct,
        oosParentLocalPct,
        oosLocalImprovement,
        oosStructuralPct,
        oosParentStructuralPct,
        oosStructuralImprovement,
        localEdge,
        structuralEdge,
        foldEligible,
        foldPass
    );

    if(foldEligible)
        eligibleCount++;

    if(foldPass)
        passCount++;

    double oosLocalWilson =
        WilsonLower95(oosStat.localReversal, oosLocalDecisive
    );
    double oosParentLocalWilson =
        WilsonLower95(oosParent.localReversal,
oosParentLocalDecisive);
    double oosStructuralWilson =
        WilsonLower95(oosStat.structuralOpposite,
oosStructuralDecisive);
    double oosParentStructuralWilson =
        WilsonLower95(

```

```

        oosParent.structuralOpposite,
        oosParent.structuralDecisive
    );

    double oosAvgMFE = Average(oosStat.sumMFE,
oosStat.countMFE);
    double oosAvgMAE = Average(oosStat.sumMAE,
oosStat.countMAE);
    double oosRatio = SafeRatio(oosAvgMFE, oosAvgMAE);

    int trainFirstSegment = selection.foldIndex;
    int trainLastSegment =
        selection.foldIndex + TrainingSegments - 1;
    int oosSegment = selection.foldIndex + TrainingSegments;

    string line = "";

    AppendField(line, IntegerToString(selection.foldIndex + 1
));
    AppendField(line, IntegerToString(trainFirstSegment + 1
));
    AppendField(line, IntegerToString(trainLastSegment + 1));
    AppendField(line, DateTimeText(g_segmentStart[
trainFirstSegment]));
    AppendField(line, DateTimeText(g_segmentEnd[
trainLastSegment]));
    AppendField(line, IntegerToString(oosSegment + 1));
    AppendField(line, DateTimeText(g_segmentStart[oosSegment
]));
    AppendField(line, DateTimeText(g_segmentEnd[oosSegment
]));
    AppendField(line, IntegerToString(selection.foldRank));
    AppendField(line, candidate.patternID);
    AppendField(line, parent.patternID);
    AppendField(line, selection.selectionReason);
    AppendField(line, selection.reliabilityTier);
    AppendField(line, candidate.patternType);
    AppendField(line, "TRADING_TIME");
    AppendField(line, Symbol());
    AppendField(line, TimeframeToString(Period()));
    AppendField(line, ZoneTypeName(candidate.zoneIndex));
    AppendField(line, AgeBucketName(candidate.ageIndex));
    AppendField(
        line,
        candidate.touchIndex >= 0
        ? TouchBucketName(candidate.touchIndex)
        : "ALL"
    );
    AppendField(
        line,
        candidate.depthIndex >= 0
        ? DepthBucketName(candidate.depthIndex)

```

```

        : "ALL"
    );
AppendField(
    line,
    candidate.approachIndex >= 0
    ? ApproachBucketName(candidate.approachIndex)
    : "ALL"
);
AppendField(
    line,
    candidate.heightIndex >= 0
    ? HeightBucketName(candidate.heightIndex)
    : "ALL"
);
AppendField(
    line,
    candidate.sessionIndex >= 0
    ? SessionBucketName(candidate.sessionIndex)
    : "ALL"
);

AppendField(line, IntegerToString(trainStat.samples));
AppendField(line, IntegerToString(trainLocalDecisive));
AppendField(line, IntegerToString(trainStat.localReversal
));
AppendField(line, DoubleOrBlank(trainLocalPct, 4));
AppendField(line, DoubleOrBlank(
selection.trainLocalWilson, 6));
AppendField(line, IntegerToString(trainParent.samples));
AppendField(line, IntegerToString(
trainParentLocalDecisive));
AppendField(line, DoubleOrBlank(trainParentLocalPct, 4));
AppendField(line, DoubleOrBlank(
selection.parentLocalWilson, 6));
AppendField(
    line,
    SignedDoubleOrBlank(selection.localImprovement, 4,
true)
);
AppendField(line, IntegerToString(trainStructuralDecisive
));
AppendField(line, IntegerToString(
trainStat.structuralOpposite));
AppendField(line, DoubleOrBlank(trainStructuralPct, 4));
AppendField(line, DoubleOrBlank(
selection.trainStructuralWilson, 6));
AppendField(line, IntegerToString(
trainParentStructuralDecisive));
AppendField(line, DoubleOrBlank(trainParentStructuralPct,
4));
AppendField(line, DoubleOrBlank(
selection.parentStructuralWilson, 6));

```

```

AppendField(
    line,
    SignedDoubleOrBlank(selection.structuralImprovement, 4
, true)
);
AppendField(line, DoubleOrBlank(trainAvgMFE, 6));
AppendField(line, DoubleOrBlank(trainAvgMAE, 6));
AppendField(line, DoubleOrBlank(trainRatio, 6));
AppendField(line, DoubleToString(selection.finalScore, 6
));

AppendField(line, IntegerToString(oosStat.samples));
AppendField(line, IntegerToString(oosLocalDecisive));
AppendField(line, IntegerToString(oosStat.localReversal
));

AppendField(line, DoubleOrBlank(oosLocalPct, 4));
AppendField(line, DoubleOrBlank(oosLocalWilson, 6));
AppendField(line, IntegerToString(oosParent.samples));
AppendField(line, IntegerToString(oosParentLocalDecisive
));

AppendField(line, DoubleOrBlank(oosParentLocalPct, 4));
AppendField(line, DoubleOrBlank(oosParentLocalWilson, 6
));

AppendField(
    line,
    SignedDoubleOrBlank(
        oosLocalImprovement,
        4,
        oosLocalPct >= 0.0 && oosParentLocalPct >= 0.0
    )
);
AppendField(line, IntegerToString(oosStructuralDecisive
));

AppendField(line, IntegerToString(
oosStat.structuralOpposite));
AppendField(line, DoubleOrBlank(oosStructuralPct, 4));
AppendField(line, DoubleOrBlank(oosStructuralWilson, 6));
AppendField(line, IntegerToString(
oosParentStructuralDecisive));
AppendField(line, DoubleOrBlank(oosParentStructuralPct, 4
));

AppendField(line, DoubleOrBlank(oosParentStructuralWilson
, 6));
AppendField(
    line,
    SignedDoubleOrBlank(
        oosStructuralImprovement,
        4,
        oosStructuralPct >= 0.0 && oosParentStructuralPct
>= 0.0
    )
);

```

```

AppendField(line, DoubleOrBlank(oosAvgMFE, 6));
AppendField(line, DoubleOrBlank(oosAvgMAE, 6));
AppendField(line, DoubleOrBlank(oosRatio, 6));
AppendField(line, YesNo(eligibleLocal));
AppendField(line, YesNo(eligibleStructural));
AppendField(line, YesNo(localEdge));
AppendField(line, YesNo(structuralEdge));
AppendField(line, YesNo(foldEligible));
AppendField(line, YesNo(foldPass));

    FileWriteString(handle, line + "\r\n");
}

FileFlush(handle);
FileClose(handle);

Print("Файл создан: ",
      TerminalInfoString(TERMINAL_DATA_PATH),
      "\\MQL4\\Files\\", OutputWalkForwardFileName);

Print("Блок 06, независимые строки: ", ArraySize(
g_selections),
      ", пригодны OOS: ", eligibleCount,
      ", подтвердились: ", passCount);

return(true);
}

//+-----

//| Поиск или создание сводки паттерна

//+-----
int FindSummary(int candidateIndex)
{
    for(int i = 0; i < ArraySize(g_summaries); i++)
    {
        if(g_summaries[i].candidateIndex == candidateIndex)
            return(i);
    }

    return(-1);
}

int AddSummary(int candidateIndex)
{
    int size = ArraySize(g_summaries);

    if(ArrayResize(g_summaries, size + 1) != size + 1)
        return(-1);
}

```

```

PatternSummary item;
item.candidateIndex = candidateIndex;
item.selectedFolds = 0;
item.eligibleFolds = 0;
item.passedFolds = 0;
item.localEligibleFolds = 0;
item.localPositiveFolds = 0;
item.sumLocalImprovement = 0.0;
item.structuralEligibleFolds = 0;
item.structuralPositiveFolds = 0;
item.sumStructuralImprovement = 0.0;
ResetStat(item.pooledPattern);
ResetStat(item.pooledParent);
item.lastSelectedFold = -1;
item.lastFoldEligible = false;
item.lastFoldPass = false;

g_summaries[size] = item;
return(size);
}

//+-----

//| Построение агрегированной OOS-сводки

//+-----
bool BuildSummaries()
{
    ArrayResize(g_summaries, 0);

    for(int i = 0; i < ArraySize(g_selections); i++)
    {
        FoldSelection selection = g_selections[i];
        int summaryIndex = FindSummary(selection.candidateIndex);

        if(summaryIndex < 0)
            summaryIndex = AddSummary(selection.candidateIndex);

        if(summaryIndex < 0)
        {
            Print("Ошибка памяти при создании OOS-сводки.");
            return(false);
        }

        PatternSummary item = g_summaries[summaryIndex];

        bool eligibleLocal;
        bool eligibleStructural;
        double localPct;
        double parentLocalPct;
        double localImprovement;

```

```

double structuralPct;
double parentStructuralPct;
double structuralImprovement;
bool localEdge;
bool structuralEdge;
bool foldEligible;
bool foldPass;

CalculateOosState(
    selection,
    eligibleLocal,
    eligibleStructural,
    localPct,
    parentLocalPct,
    localImprovement,
    structuralPct,
    parentStructuralPct,
    structuralImprovement,
    localEdge,
    structuralEdge,
    foldEligible,
    foldPass
);

item.selectedFolds++;

if(foldEligible)
    item.eligibleFolds++;

if(foldPass)
    item.passedFolds++;

if(eligibleLocal)
{
    item.localEligibleFolds++;
    item.sumLocalImprovement += localImprovement;

    if(localEdge)
        item.localPositiveFolds++;
}

if(eligibleStructural)
{
    item.structuralEligibleFolds++;
    item.sumStructuralImprovement += structuralImprovement
;

    if(structuralEdge)
        item.structuralPositiveFolds++;
}

SampleStat oosPattern =

```

```

        g_oosStats[
            FoldStatIndex(selection.foldIndex,
selection.candidateIndex)
        ];

        SampleStat oosParent =
            g_oosStats[
                FoldStatIndex(selection.foldIndex,
selection.parentIndex)
            ];

        AddStat(item.pooledPattern, oosPattern);
        AddStat(item.pooledParent, oosParent);

        if(selection.foldIndex >= item.lastSelectedFold)
        {
            item.lastSelectedFold = selection.foldIndex;
            item.lastFoldEligible = foldEligible;
            item.lastFoldPass = foldPass;
        }

        g_summaries[summaryIndex] = item;
    }

    return(true);
}

//+-----

//| Итоговая классификация независимой проверки

//+-----
string BuildOosGrade(PatternSummary &item)
{
    if(item.eligibleFolds <= 0)
        return("INSUFFICIENT_OOS");

    if(item.selectedFolds >= 2 &&
        item.eligibleFolds >= 2 &&
        item.passedFolds == item.eligibleFolds)
    {
        return("CONFIRMED_TWO_FOLDS");
    }

    if(item.selectedFolds >= 2 && item.passedFolds > 0)
        return("REPEATED_MIXED");

    if(item.selectedFolds == 1 && item.passedFolds == 1)
        return("SINGLE_FOLD_PASS");

    return("OOS_FAILED");
}

```

```

}

//+-----

//| Сводный файл по уникальным паттернам

//+-----
bool SaveWalkForwardSummary()
{
    if(!BuildSummaries())
        return(false);

    ResetLastError();

    int handle = FileOpen(
        OutputWalkForwardSummaryFileName,
        FILE_WRITE | FILE_TXT | FILE_ANSI
    );

    if(handle == INVALID_HANDLE)
    {
        Print("Ошибка создания ",
OutputWalkForwardSummaryFileName,
        ". Код ошибки: ", GetLastError());
        ResetLastError();
        return(false);
    }

    string header =

"PatternID;PatternType;AgeBasis;Symbol;Timeframe;ZoneType;" +
    "AgeBucket;TouchBucket;DepthBucket;ApproachBucket;" +

"ZoneHeightBucket;SessionBucket;SelectedFolds;EligibleFolds;" +
    "PassedFolds;PassSharePct;LocalEligibleFolds;" +
    "LocalPositiveFolds;AvgLocalImprovementPctPoints;" +
    "StructuralEligibleFolds;StructuralPositiveFolds;" +
    "AvgStructuralImprovementPctPoints;PooledOOSSamples;" +
    "PooledOOSLocalDecisive;PooledOOSLocalReversalPct;" +

"PooledParentLocalReversalPct;PooledLocalImprovementPctPoints;"
+

"PooledOOSStructuralDecisive;PooledOOSStructuralOppositePct;" +
    "PooledParentStructuralOppositePct;" +

"PooledStructuralImprovementPctPoints;PooledAvgMFE_CloseATR;" +

"PooledAvgMAE_CloseATR;PooledMFE_MAE_Ratio;LastSelectedFold;" +
    "LastFoldEligible;LastFoldPass;OOSGrade;OOSAccepted\r\n";

    FileWriteString(handle, header);

```

```

int confirmed = 0;
int repeatedMixed = 0;
int singlePass = 0;
int failed = 0;
int insufficient = 0;

for(int i = 0; i < ArraySize(g_summaries); i++)
{
    PatternSummary item = g_summaries[i];
    CandidateDef candidate = g_candidates[item.candidateIndex
];

    int pooledLocalDecisive =
        item.pooledPattern.localReversal +
        item.pooledPattern.localBreakout;

    int pooledParentLocalDecisive =
        item.pooledParent.localReversal +
        item.pooledParent.localBreakout;

    int pooledStructuralDecisive =
        item.pooledPattern.structuralOpposite +
        item.pooledPattern.structuralSameType;

    int pooledParentStructuralDecisive =
        item.pooledParent.structuralOpposite +
        item.pooledParent.structuralSameType;

    double pooledLocalPct =
        Percent(
            item.pooledPattern.localReversal,
            pooledLocalDecisive
        );

    double pooledParentLocalPct =
        Percent(
            item.pooledParent.localReversal,
            pooledParentLocalDecisive
        );

    double pooledStructuralPct =
        Percent(
            item.pooledPattern.structuralOpposite,
            pooledStructuralDecisive
        );

    double pooledParentStructuralPct =
        Percent(
            item.pooledParent.structuralOpposite,
            pooledParentStructuralDecisive
        );
}

```

```

double pooledLocalImprovement = -1.0;
double pooledStructuralImprovement = -1.0;

if(pooledLocalPct >= 0.0 && pooledParentLocalPct >= 0.0)
    pooledLocalImprovement = pooledLocalPct -
pooledParentLocalPct;

if(pooledStructuralPct >= 0.0 &&
    pooledParentStructuralPct >= 0.0)
{
    pooledStructuralImprovement =
        pooledStructuralPct - pooledParentStructuralPct;
}

double avgLocalImprovement =
    item.localEligibleFolds > 0
    ? item.sumLocalImprovement / (double)
item.localEligibleFolds
    : -1.0;

double avgStructuralImprovement =
    item.structuralEligibleFolds > 0
    ? item.sumStructuralImprovement /
        (double)item.structuralEligibleFolds
    : -1.0;

double passShare =
    item.eligibleFolds > 0
    ? 100.0 * (double)item.passedFolds /
        (double)item.eligibleFolds
    : -1.0;

double avgMFE =
    Average(item.pooledPattern.sumMFE,
item.pooledPattern.countMFE);
double avgMAE =
    Average(item.pooledPattern.sumMAE,
item.pooledPattern.countMAE);
double ratio = SafeRatio(avgMFE, avgMAE);

string grade = BuildOosGrade(item);
bool accepted =
    grade == "CONFIRMED_TWO_FOLDS" ||
    grade == "SINGLE_FOLD_PASS";

if(grade == "CONFIRMED_TWO_FOLDS")
    confirmed++;
else if(grade == "REPEATED_MIXED")
    repeatedMixed++;
else if(grade == "SINGLE_FOLD_PASS")
    singlePass++;
else if(grade == "INSUFFICIENT_OOS")

```

```

        insufficient++;
else
    failed++;

string line = "";

AppendField(line, candidate.patternID);
AppendField(line, candidate.patternType);
AppendField(line, "TRADING_TIME");
AppendField(line, Symbol());
AppendField(line, TimeframeToString(Period()));
AppendField(line, ZoneTypeName(candidate.zoneIndex));
AppendField(line, AgeBucketName(candidate.ageIndex));
AppendField(
    line,
    candidate.touchIndex >= 0
    ? TouchBucketName(candidate.touchIndex)
    : "ALL"
);
AppendField(
    line,
    candidate.depthIndex >= 0
    ? DepthBucketName(candidate.depthIndex)
    : "ALL"
);
AppendField(
    line,
    candidate.approachIndex >= 0
    ? ApproachBucketName(candidate.approachIndex)
    : "ALL"
);
AppendField(
    line,
    candidate.heightIndex >= 0
    ? HeightBucketName(candidate.heightIndex)
    : "ALL"
);
AppendField(
    line,
    candidate.sessionIndex >= 0
    ? SessionBucketName(candidate.sessionIndex)
    : "ALL"
);
AppendField(line, IntegerToString(item.selectedFolds));
AppendField(line, IntegerToString(item.eligibleFolds));
AppendField(line, IntegerToString(item.passedFolds));
AppendField(line, DoubleOrBlank(passShare, 4));
AppendField(line, IntegerToString(item.localEligibleFolds
));
AppendField(line, IntegerToString(item.localPositiveFolds
));
AppendField(

```

```

        line,
        SignedDoubleOrBlank(
            avgLocalImprovement,
            4,
            item.localEligibleFolds > 0
        )
    );
    AppendField(line, IntegerToString(
item.structuralEligibleFolds));
    AppendField(line, IntegerToString(
item.structuralPositiveFolds));
    AppendField(
        line,
        SignedDoubleOrBlank(
            avgStructuralImprovement,
            4,
            item.structuralEligibleFolds > 0
        )
    );
    AppendField(line, IntegerToString(
item.pooledPattern.samples));
    AppendField(line, IntegerToString(pooledLocalDecisive));
    AppendField(line, DoubleOrBlank(pooledLocalPct, 4));
    AppendField(line, DoubleOrBlank(pooledParentLocalPct, 4
));
    AppendField(
        line,
        SignedDoubleOrBlank(
            pooledLocalImprovement,
            4,
            pooledLocalPct >= 0.0 && pooledParentLocalPct >=
0.0
        )
    );
    AppendField(line, IntegerToString(
pooledStructuralDecisive));
    AppendField(line, DoubleOrBlank(pooledStructuralPct, 4));
    AppendField(line, DoubleOrBlank(pooledParentStructuralPct
, 4));
    AppendField(
        line,
        SignedDoubleOrBlank(
            pooledStructuralImprovement,
            4,
            pooledStructuralPct >= 0.0 &&
            pooledParentStructuralPct >= 0.0
        )
    );
    AppendField(line, DoubleOrBlank(avgMFE, 6));
    AppendField(line, DoubleOrBlank(avgMAE, 6));
    AppendField(line, DoubleOrBlank(ratio, 6));

```

```

        AppendField(line, IntegerToString(item.lastSelectedFold +
1));
        AppendField(line, YesNo(item.lastFoldEligible));
        AppendField(line, YesNo(item.lastFoldPass));
        AppendField(line, grade);
        AppendField(line, YesNo(accepted));

        FileWriteString(handle, line + "\r\n");
    }

    FileFlush(handle);
    FileClose(handle);

    Print("Файл создан: ",
        TerminalInfoString(TERMINAL_DATA_PATH),
        "\\MQL4\\Files\\", OutputWalkForwardSummaryFileName);

    Print("Блок 06, сводка: CONFIRMED_TWO_FOLDS=", confirmed,
        ", REPEATED_MIXED=", repeatedMixed,
        ", SINGLE_FOLD_PASS=", singlePass,
        ", OOS_FAILED=", failed,
        ", INSUFFICIENT_OOS=", insufficient);

    return(true);
}

//+-----
//| Проверка входных параметров
//+-----
bool ValidateInputs()
{
    if(NumberOfSegments < 3 || NumberOfSegments > 50)
    {
        Print("Ошибка: NumberOfSegments должен быть от 3 до 50."
);
        return(false);
    }

    if(TrainingSegments < 1 || TrainingSegments >=
NumberOfSegments)
    {
        Print(
"Ошибка: TrainingSegments должен быть меньше NumberOfSegments."
);
        return(false);
    }

    if(MinTrainSamples < 1 || StrongTrainSamples <
MinTrainSamples)
    {

```

```

        Print("Ошибка параметров обучающей выборки.");
        return(false);
    }

    if(MinTrainLocalDecisive < 1 ||
        MinTrainStructuralDecisive < 1)
    {
        Print(
"Ошибка: минимумы определившихся исходов должны быть больше 0."
);
        return(false);
    }

    if(MinTrainLocalWilson < 0.0 || MinTrainLocalWilson > 100.0
||
        MinTrainStructuralWilson < 0.0 ||
        MinTrainStructuralWilson > 100.0)
    {
        Print("Ошибка: пороги Уилсона должны быть от 0 до 100.");
        return(false);
    }

    if(MinTrainImprovementPctPoints < 0.0 ||
        MinOosImprovementPctPoints < 0.0)
    {
        Print(
"Ошибка: минимальное преимущество не может быть отрицательным."
);
        return(false);
    }

    if(MinOosSamples < 1 || MinOosLocalDecisive < 1 ||
        MinOosStructuralDecisive < 1)
    {
        Print("Ошибка параметров OOS-выборки.");
        return(false);
    }

    if(MaxSelectedPatternsPerFold < 0)
    {
        Print(
"Ошибка: MaxSelectedPatternsPerFold не может быть отрицательным."
);
        return(false);
    }

    if(WeightLocalWilson < 0.0 || WeightStructuralWilson < 0.0
||
        WeightMfeMae < 0.0 || WeightImprovement < 0.0)
    {
        Print(
"Ошибка: веса рейтинга не могут быть отрицательными.");
        return(false);
    }

```

```

if(AsiaEndHour < 0 || AsiaEndHour > 23 ||
    LondonEndHour < 0 || LondonEndHour > 23 ||
    NewYorkEndHour < 0 || NewYorkEndHour > 24 ||
    AsiaEndHour > LondonEndHour ||
    LondonEndHour > NewYorkEndHour)
{
    Print("Ошибка границ торговых сессий.");
    return(false);
}

return(true);
}

//+-----

//| Запуск скрипта

//+-----
void OnStart()
{
    Print(
"=====
");
    Print(
"RectangleZoneAnalyzer_06: настоящий rolling walk-forward");
    Print(
"=====
");

    if(!ValidateInputs())
        return;

    if(!FindTouchRange())
        return;

    if(!BuildSegments())
        return;

    if(!BuildCandidates())
    {
        Print("Ошибка создания пространства кандидатов.");
        return;
    }

    if(!AllocateStatistics())
        return;

    if(!AggregateRollingWindows())
        return;

    if(!SelectPatterns())
        return;

```

```

if(!SaveWalkForwardDetails())
    return;

if(!SaveWalkForwardSummary())
    return;

Print(
"=====
");
Print("Блок 06 завершён успешно.");
Print("Отобрано строк по всем окнам: ", ArraySize(
g_selections));
Print("Уникальных паттернов в OOS-сводке: ", ArraySize(
g_summaries));
Print(
"=====
");
};
}

```

//+-----